

Chetna: 7-Day Work Plan

AI flood early-warning system | Prototype with a simulated sensor feed (no hardware)

Problem statement IS-12 | Sponsor: Ernst & Young (EY)

Prototype in one line: rain forecast + terrain + simulated sensors → AI risk map → official approves → citizen gets an SMS/WhatsApp alert with a safe route to the nearest safe location.

Roles

Label	Role	Owns
F1	Frontend lead	Dashboard map, time slider, alert centre, demo button
F2	Frontend support	Citizen view (safest-location finder), at-risk lists, route display, slides, outsider testing
B1	Backend / data	Data download, grid, database, forecast fetch, pipeline, routing
B2	Backend / alerts	Sensor simulator, sensor correction, Twilio/Telegram alerts, failure tests
M1	AI / ML	Static risk, labels, model, explanations, backtest and baselines

If your team has only 4 people: merge F2 into F1 and B2 (F1 takes the citizen view and route display; B2 takes slides and outsider testing).

Working rules

- One GitHub repo, one branch per person. Merge into main every evening.
- 15-minute standup each morning, 30-minute integration check each evening.
- Never wait on someone: use dummy data that matches the agreed format until the real piece is ready.
- If a task slips, cut from the nice-to-have list at the end. Never cut the end-to-end demo.

Data contract (agree on Day 1, morning)

Item	Agreed format
Grid	File with cell_id + geometry, about 200 m cells, one metric CRS for everyone
Risk table	cell_id, timestamp, horizon (1/3/6), level (Low/Moderate/High/Severe), probability
Sensor table	sensor_id, timestamp, level_cm, status, source (simulated or real)
Functions	get_risk(horizon), safe_route(lat, lon), send_alert(zone), run_pipeline(), get_sensor_readings()

How the simulated sensors work (kept simple)

- 8-10 virtual sensors placed at low points and underpasses found by M1 and B1.
- Each sensor's water level rises with the rainfall input and drains slowly afterwards.
- Readings go into the same sensor table a real device would use, with source = simulated.
- The correction code reads only that table, so real sensors can replace the simulator later without changing anything else.
- Noise and dropouts are optional extras for Day 6 if time is left.
- Be open about it: sensors are simulated (the brief allows this). Accuracy numbers come from the backtest against real past flooding, not from the sensors.

Week at a glance

Day	Theme	Main output
1	Setup and data freeze	Repo, grid, data downloaded, alert test message received
2	Static risk, forecast, simulator	Static risk map on dashboard; simulated sensors writing to DB
3	Prediction model	Model risk on map with +1h/+3h/+6h slider; sensor correction
4	Routing, alerts, explanations	Approve button sends a real SMS/WhatsApp message with a safe route
5	Backtest and end-to-end	Results table vs baselines; full demo runs without touching code
6	Polish and freeze	Slides, README, failure tests; feature freeze at day end
7	Rehearse and record	3 timed run-throughs and a backup demo video

Test schedule (built into the plan)

When	What is tested	Who
Days 3-4	Unit tests as each piece is finished (forecast fetch, grid, simulator, correction, routing, templates)	Each owner
Day 5	Backtest vs baselines; first full end-to-end run	M1, B1, all
Day 6	Failure tests (internet off, Twilio down, silent sensor); fresh-laptop install test	B2, F2
Day 7	Timed rehearsals only; no new features	All

Day 1: Setup and data freeze

Goal: Everyone can run the repo, the data is downloaded, and the data contract is agreed.

Role	Tasks	Done when	✓
All	Choose the pilot city (documented waterlogging, good OpenStreetMap coverage). Set up the repo and a shared conda environment. Agree the data contract above.	Everyone can clone and run the repo	
B1	Download the DEM and OSM data (roads, hospitals, schools, shelters). Build the ~200 m grid and freeze the CRS. Create the SQLite tables.	cells file shared with the team	
B2	Set up a Twilio account and WhatsApp sandbox and send a test message. Create a Telegram bot as backup. Test that GIS libraries install.	Test message received on a phone	
M1	Collect 5-10 known waterlogging spots (news, complaints). Download past rainfall. Pick 1-2 heavy-rain events for the backtest.	Hotspot list and event dates saved in data/	
F1	Streamlit skeleton with a Folium map of the city and the page layout (map, lists, alert centre).	Empty map loads	
F2	Wireframes for the alert centre and citizen view (safest-location finder). Slide outline. Draft the English and Hindi message text.	Wireframes and draft messages shared	

End-of-day checkpoint: Everyone has the same grid, the same data and a working repo. Twilio or Telegram delivers a test message.

Day 2: Static risk, forecast and simulated sensors

Goal: Static risk map on the dashboard and a simulated sensor feed writing to the database.

Role	Tasks	Done when	✓
B1	Write the Open-Meteo forecast fetcher for the next 6 hours and cache every response to JSON. Save forecasts in the database.	6-hour forecast stored in DB	
B2	Sensor simulator v1: 8-10 virtual sensors at low points. Water level rises with rainfall and drains slowly. Write to the sensor table with source = simulated.	Readings appear in the sensor table	
M1	Static risk: elevation, slope, flow accumulation and imperviousness per cell, combined into one vulnerability score.	cells.vulnerability filled for every cell	
F1	Load the grid and colour it by static vulnerability. Add a layer toggle.	Static risk map visible	
F2	At-risk list (hospitals and schools overlaid on cells). Citizen view form stub. Sensor status panel with dummy data.	Lists render	

End-of-day checkpoint: The dashboard shows the static risk map, and the sensor table is filling with simulated readings.

Day 3: Prediction model

Goal: Model gives Low/Moderate/High/Severe per cell at +1h, +3h and +6h, corrected by sensors.

Role	Tasks	Done when	✓
M1	Build the runoff-proxy labels and calibrate against the known hotspots. Train XGBoost. Write risk per cell and horizon to risk_predictions. A rough first version is fine.	Model gives a risk level per cell	
B1	Write run_pipeline(): fetch forecast, build features, predict, write to the DB. Run it on an hourly loop.	One command refreshes all risk	
B2	Sensor correction: if a sensor reads above what the model expects, raise risk for nearby cells (weighted blend). Write unit tests for it.	Corrected risk stored; tests pass	
F1	Risk heatmap with the +1h/+3h/+6h time slider. Sensor markers coloured by status.	Map changes with the slider	
F2	Show sensor readings and a source label (simulated). Placeholder for the why-flagged panel. Finish the message templates in English and Hindi.	Templates ready and rendering	

End-of-day checkpoint: The map shows real model predictions at three time horizons, and a high sensor reading visibly raises nearby risk.

Day 4: Routing, alerts and explanations

Goal: Click Approve and a real SMS/WhatsApp message with a safe route to the nearest safe location reaches a phone.

Role	Tasks	Done when	✓
B1	Routing (safest-location engine): build the OSM road graph, penalise or remove roads in High/Severe cells, run A* to the nearest shelter. Write safe_route(lat, lon). Handle the no-safe-route case.	Route returned and avoids red cells	
B2	Alert service: fill templates, send via Twilio (Telegram fallback), keep an alert log, de-duplicate, and only send after approval.	send_alert(zone) delivers a message	
M1	Turn feature importance or SHAP into one line of text per zone, for example: low ground + heavy forecast rain.	Why-flagged text available	
F1	Alert centre page: message preview, Approve and Reject buttons, alert log.	Approve triggers the send	
F2	Draw the safe route on the map. Citizen view: enter a location and see risk level plus the safest route out.	Route visible on screen	

End-of-day checkpoint: Clicking Approve on the dashboard delivers a message to a real phone, and the route in the message matches the map.

Day 5: Backtest and end-to-end run

Goal: Measured results against baselines, and a full demo that runs without touching code.

Role	Tasks	Done when	✓
M1	Replay the past event hour by hour. Compare our model against the rain-threshold rule and the static-map-only baseline (recall, false alarms, lead time). Add an ablation: with and without sensor correction. Hold out whole wards.	One results table	
B1	Add replay mode to the pipeline. Build the backend for the Simulate heavy rain scenario: a storm profile that raises rainfall, sensors and risk in sequence.	Scenario runs on demand	
B2	Upgrade the simulator to follow the storm profile. Run end-to-end tests. Add error handling and cached fallbacks when an API is down.	Demo works with the internet off	
F1	Simulate heavy rain button that drives the whole demo. Auto-refresh. Results table page.	One click runs the demo	
F2	Run the demo as a stranger would and log every bug. Capture screenshots. Build the results slide.	Bug list shared	

End-of-day checkpoint: The full demo runs start to finish without touching code, and the results table is filled with measured numbers.

Do not quote accuracy numbers to the team or judges before this backtest is done. Report what you measured, with its limits.

Day 6: Polish, failure tests and feature freeze

Goal: A presentable, robust demo, finished slides, and no new features after today.

Role	Tasks	Done when	✓
F1	UI polish: colours, legend, loading states, clean layout. Fix bugs from the Day 5 list.	Looks presentable	
F2	Finish the 7-8 slides (problem, existing gaps, solution, flow, demo, results, impact, limits). Check the Hindi text. Run the fresh-laptop install test using only the README.	Slides done; clean install works	
B1	Fix backend bugs first. Only if everything is stable, add the optional river mode (upstream gauge trigger, travel time = distance / speed).	Bug list closed	
B2	Failure tests: internet off, Twilio down (Telegram takes over), one sensor goes silent (shown as offline, not zero). Write README and requirements.txt. Optional: sensor noise and dropouts, Firebase push.	All failure cases handled	
M1	Freeze the model and save the artefacts. Write the how-the-model-works slide. Prepare answers on accuracy, labels and limits.	Q&A notes ready	

End-of-day checkpoint: Feature freeze. From here on, only bug fixes and rehearsal.

Day 7: Rehearse and record

Goal: A calm, well-practised 2-3 minute demo with backups ready.

Role	Tasks	Done when	✓
All	Run the full pitch 3 times in a row with a timer. Fix only blocking bugs.	3 clean runs in a row	
F1	Runs the live demo. Keeps the backup laptop ready.	Demo runs from a fresh start	
F2	Leads the pitch and slides. Records the backup demo video together with B2.	Backup video saved	
M1	Answers model and accuracy questions, including the limits.	Rehearsed Q&A	
B1	Answers data and routing questions. Saves an offline copy of data and cached API responses.	Offline copy works	
B2	Answers alert and sensor questions. Checks the phone that receives the SMS and the Twilio balance.	Phone and credits checked	

End-of-day checkpoint: Backup video recorded, offline data saved, devices charged, and everyone knows their part.

Demo script (2-3 minutes)

- 1 Show the city map on a calm day, all green.
- 2 Click Simulate heavy rain: rainfall rises, simulated sensors update, low-lying zones turn red with a 3-hour warning.
- 3 Show the at-risk hospitals and schools and the why-flagged reason.
- 4 Click Approve: the message arrives on a phone on screen.
- 5 Show the safe route avoiding flooded roads — the safest location it leads to.
- 6 Show the results table against the rain-threshold baseline.
- 7 State the limits: simulated sensors, estimated drain data, proxy-based labels.

What to say about the simulated sensors

Sensors are simulated in this prototype, which the brief allows. In a real deployment the same pipeline reads low-cost sensors installed at underpasses and waterlogging spots. Swapping the data source does not change the rest of the system.

If you fall behind: what to cut

Priority	Items
Must have (never cut)	Risk map with time slider, simulated sensors with correction, Approve button, alert delivery, safe route to the nearest safe location, baseline comparison table
Should have	Feature-importance or SHAP explanations, the with/without sensor ablation, Hindi messages
Nice to have (cut first)	River flash-flood mode, web push, sensor noise and dropouts, extra languages

Risks and fixes

Risk	Fix	Owner
GIS libraries will not install	Use conda; test on Day 1	B1
Twilio WhatsApp sandbox issues	Use SMS or Telegram as fallback	B2
No real drain data	Use DEM and OSM proxies and say so openly	M1
Live APIs fail during the demo	Cache responses; keep a backup video	B1, B2
Weak labels	Calibrate against known hotspots; show uncertainty	M1
Scope creep	Freeze features at the end of Day 6	All

Limits to state in the pitch

- Sensors are simulated (allowed by the brief).
- Drain capacity is estimated, not surveyed.
- Labels are proxy-based and validated against known hotspots.
- It predicts rain-driven flooding, not glacier or dam collapses.
- If phone networks fail, SMS fails too; a real deployment would add sirens or loudspeakers.